Ejercicio 06: VAE, evaluación y Model Card

INF-8239 Ciencia de Datos II, Unidad 03 (U03.E06)

Estudiante	Roque Alexis Ventura Vallejo
Matrícula	100805873
Repositorio	github.com/RoqueVentura/UASD-Maestria-Ciencia-De-Datos, Ciencia de datos II/Unidad III/Ejercicio 06
Evidencia	U03_E06_Evidencias_Roque_Ventura.pdf (abrir en GitHub)
Base	LAB10, proyecto INF8239_U03_Generativos_Proyecto_Base (uv, Python 3.12.13, TensorFlow 2.21.0 en CPU)

1. Evidencia de ejecución del laboratorio

Ejecuté LAB10 en mi equipo, desde VS Code, el 21 de octubre de 2026. Las cuatro capturas (E1 a E4), cada una con lo que muestra y cómo la interpreto, están en el anexo enlazado en la fila «Evidencia» de la tabla anterior.

2. Datos, licencia y particiones

Usé **Fashion-MNIST** (Xiao, Rasul y Vollgraf, 2017): 70,000 imágenes de prendas de 28×28 en escala de grises, 10 clases y licencia MIT. Keras lo descarga en *~/.keras/datasets*; el repositorio no lo contiene. El proyecto no usa ni redistribuye MovieLens. El preprocesamiento normaliza los píxeles a [0, 1] y *validate_images* rechaza formas distintas de (28, 28, 1) o valores fuera de rango.

Partición	Imágenes	Uso
Entrenamiento AE	54,000	Ajuste del AE (validation_split = 0.1 toma las últimas 6,000)
Validación AE	6,000	EarlyStopping (paciencia 2) y curva de validación
Entrenamiento VAE	60,000	Ajuste del VAE; no tiene partición de validación
Prueba	10,000	Solo evaluación final: métricas, panel, vecinos e interpolación

Las clases están balanceadas: 6,000 por clase en entrenamiento y 1,000 en prueba (verificado en el notebook).

3. Baselines AE y VAE y reproducibilidad

Los dos baselines son los del laboratorio; no cambié su arquitectura, épocas ni pérdida.

Modelo	Arquitectura	Pérdida	Latente	Parámetros
AE	Flatten → Dense(16, relu) → Dense(784, sigmoid)	BCE	16	25,888
VAE	Codificador: Dense(128) → μ, log σ² (2). Decodificador: Dense(128) → Dense(784, sigmoid)	BCE por imagen + KL respecto de N(0, I)	2	202,516

El VAE (Kingma y Welling, 2013) muestrea $z = \mu + \exp(\tfrac{1}{2}\cdot\log\sigma^2)\cdot\varepsilon$, el truco de reparametrización que permite propagar gradientes a través del muestreo. Ambos se entrenan con Adam y batch de 256.

Reproducibilidad. El script fija la semilla 42 en random, NumPy y TensorFlow. Lo ejecuté tres veces: las métricas de prueba, el panel, la diversidad y los vecinos fueron idénticos. Solo cambió el tiempo y, en la sexta cifra decimal, la pérdida total que Keras promedia durante el VAE (342.47736 frente a 342.47733 en la época 1). Las pérdidas del AE coinciden además con las de mi captura del laboratorio (E2). El notebook carga los modelos guardados y recalcula todas las cifras: coinciden con *training_metrics.json*.

4. Tabla principal de resultados

Métricas sobre las 10,000 imágenes de prueba. El VAE reconstruye desde μ, sin muestreo. La diversidad y la memoria se calculan sobre las 16 muestras del panel y se comparan con las 16 imágenes reales del mismo panel.

Modelo	Latente	MSE por píxel	BCE por imagen	KL	Diversidad (L2 entre pares)	Distancia al vecino de entrenamiento	Parámetros	Tiempo (s)
Imágenes reales (referencia)	–	–	–	–	11.66	3.89	–	–
AE	16	0.0203	240.5	–	–	–	25,888	11.2–18.5
VAE	2	0.0331	267.2	6.34	3.67	2.32	202,516	16.7–34.4

El AE reconstruye mejor: su MSE es 39 % menor, porque su cuello de botella es 8 veces más ancho y no paga la regularización KL. Pero solo el VAE genera: el AE no tiene un espacio latente del cual muestrear con garantías. El tiempo es el rango de las cuatro ejecuciones (E2 y tres más); depende de la carga del equipo, no del modelo.

5. Reconstrucción, muestreo e interpolación

El panel no está seleccionado a mano: los 16 índices de prueba y los 16 puntos z ~ N(0, I) salen de un generador con semilla 42 y quedan registrados en *training_metrics.json*. Todas las imágenes usan la misma escala de gris, de 0 a 1.

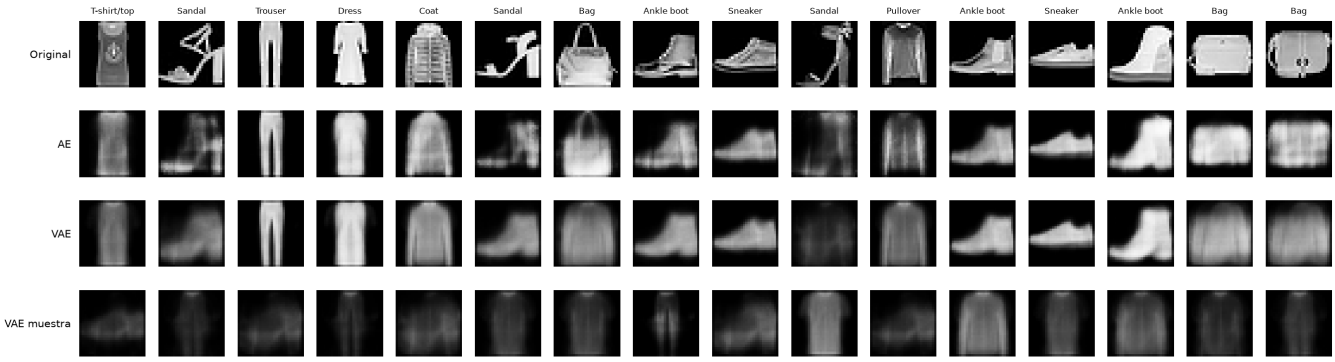


Figura 1. Original, reconstrucción AE, reconstrucción VAE y muestra nueva del VAE (16 ejemplos no seleccionados).

Reconstrucción. El AE conserva la silueta y algo de detalle, como las correas de las sandalias o el asa del bolso. El VAE, con solo 2 dimensiones, devuelve siluetas promedio: convierte sandalias en botines y bolsos en prendas superiores. Ninguno conserva texturas ni logotipos. **Muestreo.** Las muestras son calzado, prendas superiores y pantalones o vestidos reconocibles, pero borrosos y oscuros.

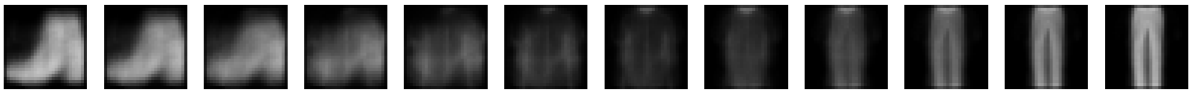


Figura 2. Interpolación lineal en el latente del VAE, de (−2, −1) a (2, 1), en 12 pasos.

Interpolación. La transición botín → prenda superior → pantalón es gradual, sin saltos: el espacio es continuo. Los pasos intermedios mezclan clases y no son prendas plausibles.

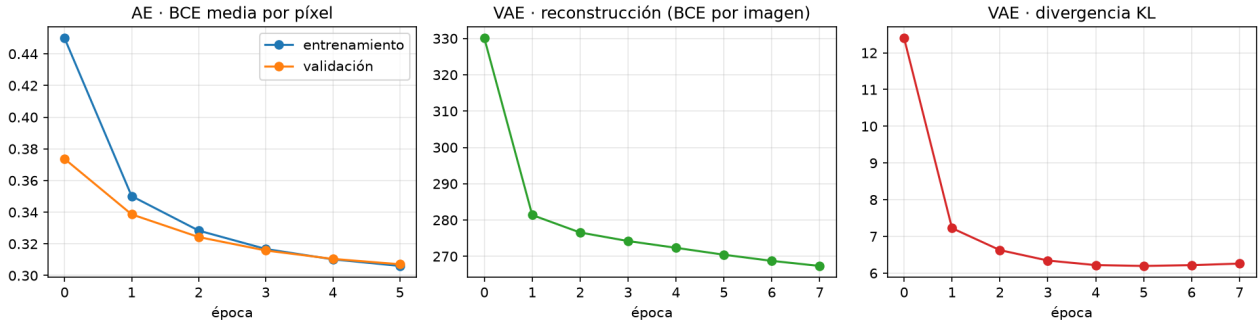


Figura 3. Curvas de entrenamiento: BCE del AE (entrenamiento y validación), reconstrucción y KL del VAE.

En el AE, entrenamiento y validación bajan juntos (0.306 frente a 0.307 al final): no hay sobreajuste. En el VAE, la reconstrucción baja de 330.1 a 267.4 y la KL cae de 12.4 a cerca de 6.2 desde la época 4: el latente se usa y no colapsa.

6. Fidelidad, diversidad y memoria

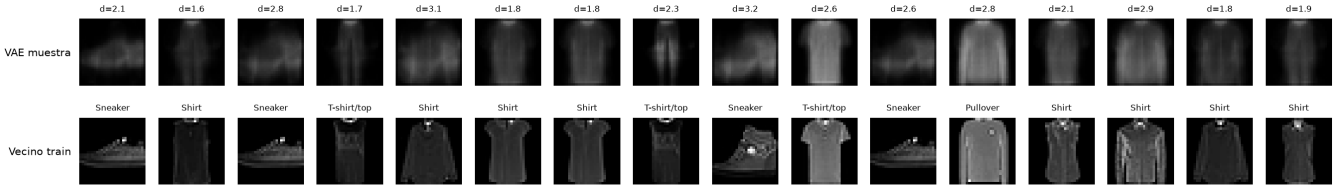


Figura 4. Cada muestra del VAE (arriba, con su distancia L2) y su vecino más cercano en entrenamiento (abajo, con su clase).

Indicador (16 imágenes)	Muestras VAE	Imágenes reales de prueba	Lectura
Fidelidad: intensidad media de píxel	0.129	0.267	Las muestras tienen la mitad de brillo: son oscuras y de bajo contraste
Diversidad: distancia L2 media entre pares	3.67	11.66	Unas 3 veces menos variadas; 3 o 4 prototipos
Memoria: distancia media al vecino de entrenamiento	2.32 (mín. 1.64)	3.89	Más cerca que las reales, pero no son copias
Vecinos distintos en entrenamiento	12 de 16	–	Uno es vecino de 3 muestras y dos lo son de 2

¿Copia o variación? Que las muestras queden más cerca del entrenamiento que las imágenes reales no indica memorización. La figura 4 muestra promedios difusos, no reproducciones: son oscuros, y en distancia L2 una imagen oscura queda cerca de prendas oscuras y lisas, como las camisas y la zapatilla que aparecen como vecinos. La L2 en píxeles no es una medida perceptual ni una prueba de privacidad.

7. Costo computacional

Concepto	AE	VAE
Parámetros	25,888	202,516 (decodificador: 101,520)
Tiempo de entrenamiento (4 ejecuciones)	11.2–18.5 s, 6 épocas	16.7–34.4 s, 8 épocas
Tamaño en disco	339 KB	853 KB (codificador 427 KB + decodificador 426 KB)
Inferencia en la app	–	Un punto latente por consulta, inmediato en CPU

Equipo: Intel Core i7-1165G7, 16 GB de RAM, Windows 11, sin GPU. El costo es bajo y no justifica una GPU para este laboratorio. Para una nube con memoria limitada basta subir el decodificador y no entrenar en cada visita.

8. Errores encontrados y cómo los resolví

Error	Causa	Solución
uv sync fallaba con *Access is denied (os error 5)* y *uv run pytest* con *uv trampoline failed to canonicalize script path*	El proyecto está en una carpeta sincronizada con OneDrive: .venv y varios archivos (dist-info, pytest.exe, streamlit.exe) tenían atributos ReadOnly y ReparsePoint	*attrib -R /S /D ".venv*"* y reinstalar pytest y streamlit con uv. La solución quedó anotada en el README
El panel mostraba los 12 primeros ejemplos de prueba	El script base usaba *x_test[:12]*; la guía pide 16 ejemplos elegidos con semilla, sin seleccionar	16 índices con *default_rng(42)*, registrados en training_metrics.json
No se guardaban curvas ni métricas de error	El script base solo registraba tiempo y parámetros	Curvas, MSE, BCE y KL sobre prueba, historial por época, diversidad y vecinos
Las muestras del VAE parecían más claras de lo que eran	*imshow* ajusta el contraste de cada imagen por separado	Escala fija de 0 a 1 y la intensidad media como medida
No se podían recalcular las métricas del VAE fuera del script	Solo se guardaba el decodificador	Se guarda también *encoder.keras*; el notebook lo usa
Los modelos y las figuras no entraban en Git	.gitignore excluía models/*.keras, reports/*.png y reports/*.json	Cambié .gitignore para versionar models/ y reports/; el dataset sigue fuera

9. Límites

(1) La diversidad y la memoria se midieron sobre 16 muestras: son indicadores, no estimaciones estables. (2) La distancia L2 en píxeles no es perceptual; no calculé métricas como FID. (3) El VAE no tiene partición de validación. (4) Las capas densas y el latente de 2 dimensiones producen imágenes borrosas y solapan las 10 clases. (5) Comprobé la reproducibilidad con una semilla, pero no la variación entre semillas. (6) El tiempo se midió en un portátil con carga variable. (7) Los modelos heredan la distribución de Fashion-MNIST: prendas de catálogo, centradas y con fondo negro.

10. Decisión final y conclusión

Pregunta del cierre	Respuesta
Resultado de reconstrucción	El AE reconstruye mejor (MSE 0.0203 frente a 0.0331). Ambos pierden texturas; el VAE confunde sandalias con botines.
Resultado de generación	El VAE genera siluetas reconocibles, pero oscuras y borrosas. El AE no se usa para generar.
Evidencia de diversidad	Diversidad 3.67 frente a 11.66 en imágenes reales; 12 vecinos distintos para 16 muestras.
Comportamiento del espacio latente	Continuo (interpolación gradual) y usado (KL estable en 6.2), pero las clases se solapan.
Costo computacional	Menos de 35 s por modelo en CPU y menos de 1 MB en disco por modelo.
Riesgo o limitación	Baja diversidad medida con pocas muestras y una distancia no perceptual; una imagen sintética no es automáticamente privada ni original.
Decisión sobre uso previsto	Solo demostración académica del espacio latente con inferencia local. No apto para reemplazar datos reales ni para uso comercial.

Decisión. El AE queda como referencia de reconstrucción y el VAE como modelo de la aplicación, porque es el único con un espacio latente muestreable. La comparación muestra que reconstruir bien no es lo mismo que generar bien: el modelo con menor error no puede generar, y el que genera lo hace con poca variedad.

11. Código, pruebas, Model Card, README y Git

Ruta	Contenido
src/inf8239_u03_gen/	data.py (normalización y validación), models.py (AE y VAE), evaluation.py (vecino más cercano, diversidad y error de reconstrucción)
scripts/	check_runtime.py (entorno) y train_ae_vae.py (entrenamiento, evaluación sobre prueba, figuras y métricas)
notebooks/01_ae_vae_fashion_mnist.ipynb	Notebook ejecutado: carga los modelos guardados, recalcula métricas, panel, interpolación, diversidad y vecinos, y verifica que coinciden con training_metrics.json
reports/	training_metrics.json, ae_vae_panel.png, interpolation.png, loss_curves.png, nearest_neighbors.png y cierre_interpretativo.md
models/	autoencoder.keras, encoder.keras y decoder.keras (el que usa la app)
MODEL_CARD.md, AI_USE_DECLARATION.md, README.md	Model Card según Mitchell et al. (2019), declaración de IA e instrucciones de reproducción
pyproject.toml, uv.lock, requirements.txt	Entorno fijado: uv como vía principal; requirements.txt exportado del lock para pip

```
uv sync --extra cpu
uv run pytest -q
uv run python scripts/train_ae_vae.py --epochs-ae 6 --epochs-vae 8
uv run jupyter nbconvert --to notebook --execute --inplace notebooks/01_ae_vae_fashion_mnist.ipynb
uv run streamlit run app/streamlit_app.py
```

Pruebas. 7 pruebas con pytest, todas pasan, y ruff no reporta errores. Comprueban el contrato de normalización, el rechazo de formas inválidas, la forma de salida del AE y del decodificador, que el vecino más cercano encuentra una copia exacta aunque esté en otro bloque, que imágenes idénticas tienen diversidad cero y que una reconstrucción perfecta tiene error cero. **Git.** El proyecto está versionado en el repositorio indicado, con models/ y reports/ como evidencia.

12. Uso de inteligencia artificial

Herramienta	Claude Code (modelo Claude Opus 5.5, de Anthropic), desde Visual Studio Code, en octubre de 2026, como apoyo durante el ejercicio.
Mi trabajo	Ejecuté LAB10 en mi equipo (E1–E4). Dirigí la revisión del proyecto según la guía y el mandato, revisé los resultados, las figuras y la decisión final, y revisé el informe.
Consultas realizadas (resumidas)	Revisión y apoyo sobre lo que ejecuté en el laboratorio LAB10; explicación de lo que me faltaba según la guía y el mandato de U03.E06; orientación para armar el entregable sin inventar evidencia, con el informe del Ejercicio 05 solo como referencia de formato.
Apoyo recibido	Trabajamos en conjunto: yo indiqué qué revisar y validé cada cambio, y la herramienta me apoyó en la ampliación del script (panel con semilla, curvas, métricas de prueba, diversidad, vecinos, guardado del codificador), el módulo evaluation.py y sus pruebas, el notebook y el diagnóstico del error de uv con OneDrive. Los borradores del README, la Model Card, el cierre, la declaración de IA y este informe los revisé y ajusté yo.
Verificaciones	Tres ejecuciones con semilla 42 dieron las mismas métricas. El notebook ejecutado coincide con training_metrics.json. Las 7 pruebas pasan y ruff está limpio. La app carga el decodificador. Comprobé que los DOI de las tres referencias resuelven en doi.org y que sus metadatos coinciden.
Correcciones realizadas	(1) La afirmación «3 muestras comparten vecino» se corrigió tras recalcularla: hay 12 vecinos distintos para 16 muestras. (2) «Muestras oscuras» se respaldó con la intensidad media (0.129 frente a 0.267). (3) Los tiempos se reportan como rango de las ejecuciones, no como un solo valor.
Responsabilidad	Revisé el contenido final, puedo explicar cada fragmento del código y asumo la responsabilidad de lo entregado.

13. Fuentes

Kingma, D. P., & Welling, M. (2013). *Auto-encoding variational Bayes* (arXiv:1312.6114). arXiv. https://doi.org/10.48550/arXiv.1312.6114

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency* (pp. 220-229). https://doi.org/10.1145/3287560.3287596

Xiao, H., Rasul, K., & Vollgraf, R. (2017). *Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms* (arXiv:1708.07747). arXiv. https://doi.org/10.48550/arXiv.1708.07747